Three-Class Sentiment Classification of Tweets Using Word and Character TF-IDF

Dattu Nayak Paltya
BCSI Programme, INTI International University, Malaysia
PRG4206 Machine Learning · Student ID I26043443

Abstract—Social-media posts are short, informal, and often ambiguous, which makes reliable sentiment classification difficult. This project develops and evaluates a reproducible three-class classifier for negative, neutral, and positive tweets using the sentiment task in the TweetEval benchmark. The system combines word- and character-level term frequency–inverse document frequency (TF-IDF) features with Multinomial Naive Bayes, Logistic Regression, and Linear Support Vector Machine (SVM) models. Hyperparameters are selected by stratified three-fold cross-validation on the official training split, and the model family is selected on the separate validation split. The official test split is reserved for the final evaluation. In the measured run, Linear SVM was selected by validation macro-F1 of 0.676 and achieved 61.9% test accuracy and 61.4% test macro-F1 on 12,284 examples. Logistic Regression achieved a slightly higher test score but was not substituted after test inspection. The prototype includes dataset exploration, integrity checks, model explanations, a local interactive dashboard, and reproducible outputs. Results show useful but limited signal: predictions remain imperfect and do not establish reliable performance on current social-media posts.

Keywords— sentiment analysis, text classification, TweetEval, TF-IDF, support vector machine, cross-validation, social media.

I. INTRODUCTION

People and organizations publish an enormous amount of short text on social platforms. A team trying to understand reactions to a product update, public service, or campaign may not be able to read every post individually. Sentiment analysis offers one way to summarize the tone of a collection of messages. It is useful as a triage or exploratory tool: it can help identify examples for human review or highlight broad patterns that deserve further investigation. It should not be treated as a substitute for human judgment.

Tweet sentiment is a particularly challenging classification problem. A post may contain only a few words, an emoji, a hashtag, an abbreviation, or a spelling variant. A phrase such as “that was sick” can be positive in one context and negative in another. Sarcasm can reverse the apparent meaning of a sentence. Even annotators may reasonably disagree about whether a short post is neutral or expresses a mild opinion. These characteristics make the task more difficult than simply searching for positive and negative words.

This project addresses a bounded question: how well can classical machine-learning models classify a tweet into negative, neutral, or positive sentiment when evaluated on the fixed TweetEval sentiment benchmark? The project does not attempt to infer a user's personality, intent, political affiliation, or mental state. It does not measure sentiment intensity, identify a target entity, or claim to describe all users of a platform. The scope is limited to a single benchmark and its labels.

The motivation for choosing the task is practical and methodological. It is an example of a real-world text analytics problem with a public benchmark, a clear categorical target, and several well-established approaches that can be compared under a common split. The benchmark supplies fixed training, validation, and test partitions, making it possible to avoid the inappropriate random splitting used for some ordered or grouped data. A compact text classifier can also be serialized and exposed through a working interface, so the project demonstrates more than a theoretical model.

The objectives are:

1. obtain and describe the public TweetEval sentiment data and its provenance; 2. inspect the classes and text, and document data-quality checks before modeling; 3. construct word and character TF-IDF features without allowing validation or test vocabulary to leak into training; 4. compare a majority baseline and at least three machine-learning models; 5. tune model hyperparameters with stratified cross-validation, select a model family using validation macro-F1, and preserve the test split for final evaluation; 6. explain model behavior with linear feature coefficients, quantify errors, and provide a reproducible prototype; and 7. state limitations and responsible-use boundaries based on the measured results.

The contribution is an end-to-end, inspectable benchmark implementation rather than a new state-of-the-art sentiment model. It joins a disciplined evaluation protocol with a clear baseline, several interpretable classical approaches, cross-validation, paired uncertainty estimates, and an interactive local demonstration. The central finding is deliberately modest: the selected model performs substantially better than predicting one class for every tweet, but its held-out accuracy is about 62%, not a level that supports high-confidence automated decisions.

II. LITERATURE REVIEW

A. Sentiment analysis and machine-learning foundations

Sentiment analysis has been studied as a form of opinion mining for many years. Pang and Lee describe the broader problem of identifying opinions, subjective language, and polarity, while emphasizing that the task depends on the text source and the question being asked [7]. Early supervised sentiment research showed that machine-learning methods could learn polarity from labeled text rather than relying only on a hand-written word list [8]. These foundations remain relevant, but modern social-media writing introduces abbreviations, platform conventions, emoticons, and quickly changing slang.

A sentiment label is not the same as a complete interpretation of an utterance. A classifier trained to predict polarity does not necessarily identify who or what the writer is discussing, whether the writer is being sarcastic, or how strongly the opinion is held. This distinction matters when

describing practical value: a model can assist with large-scale sorting and exploratory summaries, but it should not be presented as a dependable interpreter of every individual message.

VADER was designed specifically for social-media sentiment and combines a sentiment lexicon with rules for features such as punctuation, capitalization, and degree modifiers [9]. It is a useful reminder that online writing includes signals not always captured by standard bag-of-words models. In the present project, the methods are deliberately restricted to the classical ML models required for a controlled comparison; a lexicon-based system could be added as a further baseline if its output is mapped consistently to the benchmark's three classes.

B. TweetEval and the sentiment source

Barbieri et al. introduced TweetEval as a unified benchmark for several tweet-classification tasks, providing standard splits and a common evaluation framework [1]. The sentiment task is a three-class problem with labels negative, neutral, and positive. The dataset card reports 45,615 training, 2,000 validation, and 12,284 test examples, for 59,899 examples in total [2]. Using the benchmark's official partitions makes the experiment reproducible and allows discussion against published TweetEval work.

The apparent recency of the benchmark must be distinguished from the age of its underlying posts. The TweetEval paper identifies its sentiment data as originating from SemEval-2017 Task 4, Subtask A, and notes that the underlying sentiment collection draws on earlier SemEval data [1], [3]. TweetEval itself was published in 2020, which lies within the assignment's stated 2020–2026 benchmark window. However, the posts were not all newly collected in 2020. This report therefore treats the benchmark release and corpus collection as separate dates. The lecturer should confirm that a benchmark released in 2020 satisfies the recency requirement when its source posts are older.

The benchmark card marks the dataset license as unknown, while the official repository cautions that restrictions can apply to component datasets and Twitter content [2], [4]. This project avoids claiming that the tweets are unrestricted or under a particular permissive license. Researchers should follow the source dataset terms and platform policies and should avoid redistributing raw content beyond the terms that apply.

C. Later TweetEval studies

The original TweetEval benchmark reports comparative systems, and its repository maintains results for the test set [1], [4]. The public leaderboard has included classical baselines such as SVM and FastText, as well as transformer models such as RoBERTa and BERTweet. Those results provide useful context, but they should not be compared as though every implementation, pre-processing choice, checkpoint, and metric were identical.

Loureiro et al. present TimeLMs, diachronic language models trained on Twitter data, and explicitly report TweetEval test results in their evaluation [5]. Their study is particularly relevant because it compares classical systems and transformer approaches on the benchmark and makes a model-selection distinction between validation and test evaluation. It demonstrates that results depend not only on the classifier architecture but also on the pretraining data and the temporal relationship between training material and evaluation posts.

Barbieri et al. introduce XLM-T, multilingual language models for Twitter, and report evaluation within the TweetEval framework [6]. The study is relevant to social-media adaptation and the effect of pretraining on platform-specific text. The experimental materials inspected for this project do not establish that every score can be aligned exactly to the unchanged official sentiment test partition. Therefore, XLM-T is discussed as work using the benchmark, while direct numeric equivalence is not assumed.

Huang and Usbeck revisit supervised contrastive learning for microblog classification and evaluate on TweetEval subtasks [10]. Their work is a recent example of approaches that use representation learning and training objectives beyond conventional TF-IDF classification. The accessible experimental summary confirms TweetEval usage but does not, by itself, settle every split and preprocessing detail needed for a strict numerical comparison. This report treats it as evidence of recent benchmark usage and an alternative methodological direction, not as an automatically interchangeable score.

BERTweet is a Twitter-pretrained language model for English tweets [11]. It provides useful context for why domain-specific pretraining can outperform features learned only from a labeled training set. However, a later leaderboard entry for a model does not prove that the original BERTweet paper evaluated on the exact TweetEval test set. Accordingly, BERTweet is cited as relevant prior modeling work and not misattributed as a direct result in this experiment.

D. Research gap and position of this project

The selected benchmark has strong published coverage, but several reporting issues remain important for a student-scale comparison. First, studies can share a dataset name without using precisely the same split or evaluation choices. Second, a single accuracy value can hide uneven class performance. Third, many comparisons emphasize a larger pretrained model without showing how much a transparent classical baseline achieves under a leakage-controlled feature pipeline.

This project narrows the gap by reporting the official split sizes, training-set cross-validation scores, validation-based model selection, per-class test results, and a test-set confusion matrix. Word and character n-grams are fitted inside each cross-validation pipeline, so fold validation data does not determine the vocabulary. The experiment is not claimed to close the general research gap or improve the benchmark leaderboard. It provides a reproducible local baseline with explicit limits.

E. Prior-work comparison and comparability

Table I summarizes relevant work and how it is used in this report. The entries distinguish direct evidence of benchmark use from exact split equivalence. Official benchmark leaderboard values are useful reference points, but are not substituted for results reproduced by this project. In particular, the project reports accuracy and macro-averaged precision, recall, and F1 from its own saved predictions.

Published values should be compared only after checking that the same test partition and metric definition are used.

TABLE I. RELATED STUDIES AND COMPARISON SCOPE

Study	Main approach	TweetEval evidence	Use in this report
Barbieri et al. (2020) [1]	Unified benchmark; classical and transformer baselines	Defines tasks and fixed splits	Dataset definition and baseline context
Loureiro et al. (2022) [5]	Time-specific Twitter language models; classical comparisons	Explicit TweetEval test results	Closest later test-set reference
Barbieri et al. (2022) [6]	Multilingual Twitter pretrained model	Reports on TweetEval framework	Domain adaptation context; protocol needs checking
Huang and Usbeck (2024) [10]	Supervised contrastive learning	Evaluates TweetEval subtasks	Recent method context; protocol needs checking
Nguyen et al. (2020) [11]	Twitter-pretrained BERTweet model	Twitter language-model study	Pretraining context, not presumed same test run

III. DATASET COLLECTION AND UNDERSTANDING

A. Source and collection

The project obtains the sentiment configuration from cardiffnlp/tweet_eval using the Hugging Face datasets library. The dataset is downloaded on the first run and cached by the library for subsequent access. The acquisition is reproducible through the named dataset identifier and configuration; it does not call the current social-media platform API or collect new posts. This distinction is important because the evaluation measures a published benchmark, not a live stream.

Each example contains tweet text and an integer label. In the dataset card, label 0 means negative, label 1 means neutral, and label 2 means positive [2]. The model treats the text as its input and the sentiment label as the target. It does not use author identity, engagement counts, timestamps, or external user metadata. The official train, validation, and test split assignments are preserved and exported as data/train.csv, data/validation.csv, and data/test.csv; data/data.csv contains all three with a split column.

The split sizes observed during the project run are 45,615 training examples, 2,000 validation examples, and 12,284 test examples. The 59,899-example total is the sum of those benchmark partitions. The number of input features is not a fixed raw-column count after engineering: TF-IDF creates a sparse vocabulary from the training text, with a maximum of 100,000 word features and 100,000 character features before the combined representation is fitted. The actual vocabulary is learned during model fitting and is saved within the serialized pipeline.

B. Class distribution and text properties

The training split contains 7,093 negative examples (15.5%), 20,673 neutral examples (45.3%), and 17,849 positive examples (39.1%). The validation split contains 312 negative (15.6%), 869 neutral (43.5%), and 819 positive (41.0%). The test split contains 3,972 negative (32.3%), 5,937 neutral (48.3%), and 2,375 positive examples (19.3%). These figures are generated from the downloaded benchmark CSVs and are also shown in Fig. 1.

The test split has a notably different class mix from the training and validation splits. Negative examples form a larger share of test data, while positive examples form a smaller share. This is a material limitation for interpreting a validation-selected model: cross-validation on the training distribution cannot remove a distribution difference in the held-out benchmark test set. It also means a score from a randomly resampled split would not be directly comparable to the official partition.

Training tweet lengths range from 10 to 200 characters in the current snapshot. The mean is approximately 106.9 characters and the median is 113 characters. These statistics describe length, not semantic complexity. Many posts include handles, hashtags, links, punctuation, and platform-specific placeholders, which may be informative for classification but may also create shortcuts that do not transfer to other domains.

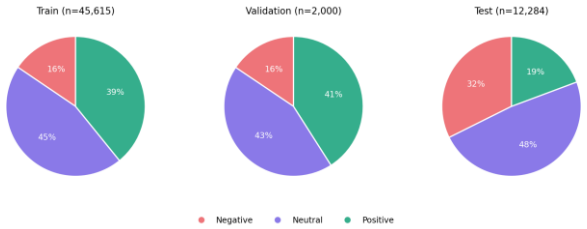


Fig. 1. Sentiment label proportions in the official train, validation, and test splits.

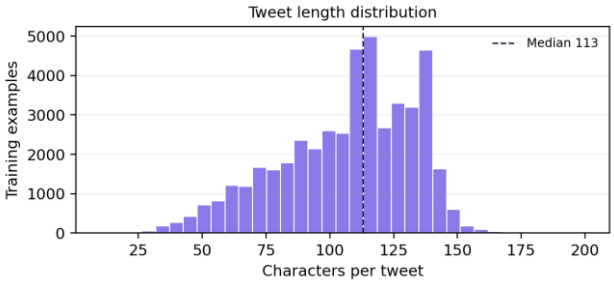


Fig. 2. Character-length distribution of the training tweets.

IV. DATA PREPROCESSING AND FEATURE ENGINEERING

A. Integrity checks and duplicate handling

The exploration checks for missing text, blank text, invalid labels, and exact duplicate (text, label) rows within each split. In the saved dataset, no missing or blank text and no invalid sentiment label were detected in train, validation, or test. The training split contains 26 exact duplicate rows; no exact duplicate rows were found in validation or test in the current checks. This report does not silently delete the training duplicates: the benchmark's official files are preserved as distributed, and the results are therefore tied to that benchmark snapshot.

Duplicate counts are not the same as proof of cross-split leakage. The project does not move, deduplicate, or randomly reassign examples across official partitions. Any broader near-duplicate investigation would need a clearly documented policy and could alter comparability with prior results. The presence of repeated records can give repeated examples additional influence during training; it is a limitation to mention rather than hide.

No imputation is required because the checked text and label fields are complete. Labels are mapped from their

documented integer encoding to human-readable names for display and reports. The original labels are retained in the exported CSVs. The text itself is not manually rewritten or stripped of punctuation. This avoids changing examples in a subjective way and allows character-level features to use useful social-media signals such as emoticons or repeated punctuation.

B. Text representation

The feature pipeline combines two TF-IDF representations. The word analyzer uses lowercased, accent-normalized unigrams and bigrams, requires a term to occur at least twice in its fitting data, and caps the word vocabulary at 100,000 features. Sublinear term-frequency scaling reduces the influence of repeated occurrences in a single message. A character-level analyzer uses character n-grams of lengths three through five within word boundaries, requires a minimum frequency of three, and caps its vocabulary at 100,000.

Word n-grams represent terms and short phrases such as “love this” or “not good.” Character fragments can recognize related spellings, contractions, informal punctuation, and parts of words that may be missed by a strict token match. TF-IDF weights indicate how informative a term is relative to its frequency across the fitting corpus. The combined word and character vectors form a sparse representation suitable for linear classifiers.

Feature construction is part of each scikit-learn pipeline. This design matters for cross-validation: the vectorizer is fitted on the training portion of each fold, not once on the full dataset before folds are created. Fitting a vocabulary on all training examples before cross-validation would allow information from fold validation text into the representation and could make CV scores optimistic. The official validation and test text are transformed only after the final fold-selected pipeline has been refitted on the complete training split.

C. Feature interpretation

Linear models provide a transparent feature-level view through their coefficients. For each class, a positive coefficient indicates that a feature contributes toward that class relative to the model's other class scores, conditional on the fitted representation. In the saved Logistic Regression explanation, positive terms include good, happy, great, best, love, and nice; negative terms include sad, not, worst, hate, and negative emoticon fragments. Neutral features include phrases and patterns that may often occur in factual or conversational messages.

These associations are not causal explanations. A term may correlate with the annotation convention or with the topics represented in this dataset. A feature can also be misleading in a particular context: “not bad” includes a negative token but can express positive sentiment. Coefficients describe global tendencies in the training data, not a guarantee that the classifier used a human-like reasoning process. The local system should therefore present its output as a prediction, not as a definitive explanation of a writer.

The final selected model is a Linear SVM, whose linear coefficients can be inspected in the same feature space. Logistic Regression coefficients are retained as a separate comparison explanation. Both models are reported as feature-based interpretability, while the distinction between the selected model and the explanatory comparator is made explicit in the outputs.

V. DATASET SPLITTING AND EXPERIMENTAL DESIGN

A. Official partitions

The project uses the supplied 45,615/2,000/12,284 train/validation/test partitions. The training split is used for model fitting and stratified three-fold cross-validation. The validation split is used to select among model families after each family has been tuned on training folds. The test split is reserved for final quantitative reporting and error analysis after that choice.

This process is distinct from creating a new random 70/15/15 split. Using the benchmark's predefined data makes external comparisons more feasible and avoids accidental movement of examples into a different partition. Class stratification applies only to cross-validation folds formed from training data; it does not alter the official validation or test data. The CV folds are shuffled with a fixed random seed of 42 to make the fold assignment reproducible.

B. Validation and test discipline

All candidate configurations are assessed by mean macro-F1 on the three training folds. The best configuration for each model family is refitted on the full training split by GridSearchCV. The validation split then selects the model family with the highest validation macro-F1. The test set is evaluated for all reported models only after this workflow, so that the benchmark results can be compared transparently. It is not used to tune a parameter or change the selected family.

The observed test score of Logistic Regression is slightly higher than that of the validation-selected SVM. This does not justify silently changing the selection after seeing the test metrics. Doing so would turn the test set into an additional validation set and would bias the final score. The report keeps the declared selection procedure and reports both test results, including the fact that the alternate model scored better on this sample.

VI. MODEL SELECTION AND TRAINING

A. Baseline and candidate algorithms

The majority baseline predicts the most frequent training class for every example. It provides a reference point for whether a classifier learned useful distinctions rather than merely exploiting the dominant class. Multinomial Naive Bayes is a probabilistic model commonly used with non-negative word features. Logistic Regression estimates linear class decision scores and supports class-weight balancing. Linear SVM learns maximum-margin boundaries in the sparse TF-IDF space, which is often effective for high-dimensional text.

The project compares these three models plus the baseline under the same text representation. The baseline does not need TF-IDF, but is included in the same result table. Logistic Regression and Linear SVM use class_weight="balanced" to adjust the training loss for class frequencies. This does not force balanced predictions, and it

does not correct the training/test class-distribution shift. Multinomial Naive Bayes uses additive smoothing.

B. Hyperparameter search

Grid search tunes Multinomial Naive Bayes smoothing alpha over 0.1, 0.5, and 1.0; Logistic Regression inverse regularization C over 0.5, 1.0, and 2.0; and Linear SVM C over 0.25, 0.5, and 1.0. The baseline has no hyperparameter search. Mean macro-F1 is the scoring function for each three-fold stratified search. Random seed 42 is set for fold creation and model components where applicable.

The best mean cross-validation macro-F1 was 0.6127 for Naive Bayes with alpha=0.1, 0.6536 for Logistic Regression with C=1.0, and 0.6513 for Linear SVM with C=0.25. The fold standard deviations were approximately 0.0032, 0.0034, and 0.0020, respectively. Those relatively small fold differences indicate similar fold-level performance within this training sample; they are not confidence intervals for performance on every future population.

C. Training procedure

For each family, GridSearchCV fits the preprocessing and classifier pipeline independently on each fold's training portion. It evaluates each candidate on that fold's held-out portion using macro-F1. After selecting the best hyperparameter, the search refits its estimator on all 45,615 training examples. The refitted model is evaluated on the official 2,000-example validation set. The family with the best validation macro-F1 is selected and serialized for inference.

The current run selected Linear SVM with validation macro-F1 of 0.6765. Logistic Regression validation macro-F1 was 0.6758, so the validation difference was very small. This near tie is useful context: the selected family is the winner under the prespecified validation metric, but the evidence does not suggest a dramatic superiority over Logistic Regression. The serialized model contains the TF-IDF vectorizers and SVM needed to transform new text consistently.

VII. SYSTEM DESIGN AND IMPLEMENTATION

A. Architecture

The implementation begins by loading the published TweetEval configuration. It validates the expected splits and columns, exports CSVs, and produces descriptive class and text-length summaries. Each model search operates on the official training split with vectorization inside the fold pipeline. Validation determines the model family. The saved model and metric artifacts support both the command-line interface and the local dashboard.

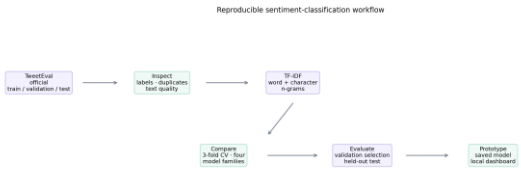


Fig. 3. System architecture from the public dataset to evaluation outputs and interactive inference.

The main components are src/model.py, which contains dataset loading, preprocessing, training, evaluation, explanation, and inference helpers; data/, which stores CSV exports; artifacts/, which stores fitted models, metrics, plots, and predictions; and app.py with web/, which provides a local interface. The Jupyter notebook, TweetEval_Sentiment_Classification.ipynb, presents the same process step by step. The project uses Python with pandas, NumPy, scikit-learn, Matplotlib, joblib, and Hugging Face datasets.

B. Prototype and reproducibility

After installing dependencies, the training workflow runs with python -m src.model. The saved model can classify a sentence using python -m src.model --predict "I really enjoyed this.". Running python app.py launches the local dashboard at http://127.0.0.1:8000. The page displays split-specific class distributions, validation/test comparisons, cross-validation results, a test confusion matrix, a browsable dataset sample, and a live text input. The UI describes the model output as a prediction, not a certainty score.

The data exports are generated from the same named public dataset and include a split column in the combined CSV. Saved artifacts include results.json, metrics.csv, cross_validation_results.csv, a per-class test report, test predictions, a confusion matrix, feature-coefficient tables, and a serialized model. Re-running the project can download the dataset again if it is not cached, so Internet connectivity is needed on the first run. The README records setup and commands.

C. System demonstration and test cases

The working prototype was checked with short illustrative inputs: “I absolutely love this, it made my day!”, “The product is okay, nothing special.”, and “Terrible service, I am very disappointed.” The observed outputs were positive, neutral, and negative, respectively. These examples demonstrate the inference path, not an independent accuracy estimate; they are chosen examples and are not a substitute for the test-set metrics.

The dashboard API was also exercised with dataset filtering, metric retrieval, and prediction requests. Automated tests cover pipeline fitting, the three-class output space, stratified cross-validation configuration, CSV export schema, dashboard metric loading, dataset browsing, and deterministic bootstrap comparisons. The final test run passed seven unit tests. Passing these checks supports implementation reliability but does not prove that the model is valid for every application.

VIII. RESULTS

A. Cross-validation and validation selection

Table II reports the best training-fold score and the official validation result for each model family. The baseline's cross-validation score is constant because it predicts only the majority training class. Logistic Regression produced the highest mean CV macro-F1 (0.6536), while Linear SVM produced the highest validation macro-F1 (0.6765) by a small margin. As specified, Linear SVM is therefore the selected family.

TABLE II. TRAINING CROSS-VALIDATION AND VALIDATION RESULTS

Model	Best parameters	CV macro-F1 mean ± SD	Validation accuracy	Validation macro-F1

Majority baseline	none	0.2079 ± 0.0000	0.4345	0.2019
Multinomial Naive Bayes	alpha=0.1	0.6127 ± 0.0032	0.6545	0.6301
Logistic Regression	C=1.0	0.6536 ± 0.0034	0.6975	0.6758
Linear SVM	C=0.25	0.6513 ± 0.0020	0.7045	0.6765

The CV and validation rankings differ slightly. Cross-validation measures average performance across folds sampled from the training distribution, whereas the official validation partition is a separate 2,000-example set. A small change in ranking is plausible, particularly when the two best models are close. It is not evidence that one metric is wrong. It reinforces why the selection rule must be declared before test inspection.

B. Held-out test performance

Table III reports the final scores on 12,284 test examples. The selected Linear SVM attained 0.6191 accuracy, macro precision 0.6081, macro recall 0.6241, and macro-F1 0.6143. Logistic Regression attained 0.6246 accuracy and 0.6223 macro-F1. Naive Bayes achieved 0.6045 accuracy and 0.6010 macro-F1. The majority baseline achieved 0.4833 accuracy and 0.2172 macro-F1.

The selected SVM improves substantially over the baseline in macro-F1, which is expected when a model predicts all three classes rather than one. Its overall accuracy remains only 61.9%, meaning that roughly 38% of test predictions are incorrect. The test macro-F1 is below the validation macro-F1, consistent with the test class distribution differing from the training and validation distributions. These results do not support using the output as an unreviewed decision.

TABLE III. OFFICIAL TEST-SET RESULTS

Model	Accuracy	Macro precision	Macro recall	Macro-F1
Majority baseline	0.4833	0.1611	0.3333	0.2172
Multinomial Naive Bayes	0.6045	0.5972	0.6083	0.6010
Logistic Regression	0.6246	0.6160	0.6336	0.6223
Linear SVM (selected)	0.6191	0.6081	0.6241	0.6143

Fig. 4 gives a visual comparison of validation and test scores, and Fig. 5 shows the selected model's confusion matrix. The macro scores average each class's precision, recall, or F1 equally. Accuracy weights examples equally and, given the larger neutral test class, can obscure weaker results on a smaller class. Reporting both types of metric gives a more complete account.

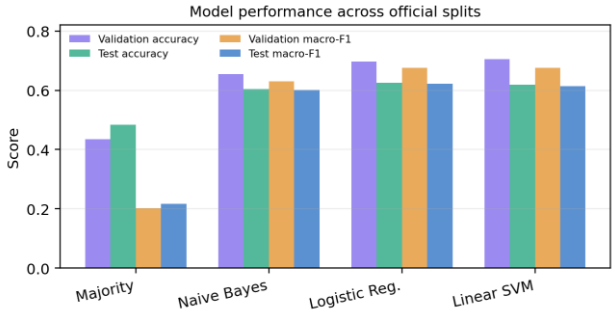


Fig. 4. Validation and official test accuracy and macro-F1 for each model.

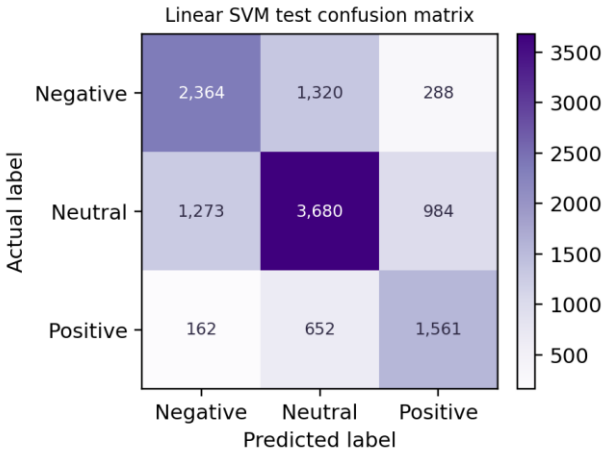


Fig. 5. Confusion matrix for the validation-selected Linear SVM on the official test split.

C. Per-class errors

For the selected SVM, negative sentiment has precision 0.6223, recall 0.5952, and F1 0.6084 over 3,972 examples. Neutral sentiment has precision 0.6511, recall 0.6198, and F1 0.6351 over 5,937 examples. Positive sentiment has precision 0.5510, recall 0.6573, and F1 0.5995 over 2,375 examples. Positive recall is higher than positive precision: the model finds a larger portion of positive examples but also labels some non-positive examples as positive. Positive class precision is the weakest of the three classes.

The errors are consistent with the limits of n-gram representations. Neutral and mildly positive or negative expressions can use similar vocabulary. Sarcasm and rhetorical questions may contradict literal word polarity. Topic-specific language can introduce accidental associations, and a small change in class distribution can change how ambiguous examples are labeled. The confusion matrix makes these errors visible but does not by itself identify their linguistic cause. A qualitative review should sample both false positives and false negatives for each class, with examples treated carefully because dataset terms may constrain redistribution.

D. Uncertainty and model comparison

The paired bootstrap resamples the same test indices for the selected SVM and each alternative, then calculates the macro-F1 difference 1,000 times. For SVM minus Logistic Regression, the observed difference is -0.0079 and the percentile 95% interval is approximately [-0.0126, -0.0032]. The interval is below zero, so on this test sample Logistic Regression's macro-F1 is higher. For the differences against the majority baseline and Naive Bayes, the intervals are positive.

This paired interval reflects sampling variation conditional on the saved test sample and prediction vectors. It does not account for differences across data sources, new collection periods, annotation policies, or parameter-search choices. It is not a test of whether a model is "good" in a practical deployment. The model selection remains Linear SVM because validation macro-F1 defined the family selection; the interval is reported to make the test comparison transparent, not to select a replacement model retrospectively.

E. Comparison with published benchmark results

The official TweetEval repository lists test-set benchmark results, including classical SVM and FastText baselines and higher values for Twitter-pretrained or transformer models [4]. The TimeLMs paper reports TweetEval test evaluations [5]. The original TweetEval study documents baselines and provides the benchmark's methodological reference [1]. The current project's SVM macro-F1 of 0.614 is lower than the SVM result commonly listed in the official leaderboard (approximately 0.629) [4]. This is a useful diagnostic, but the difference should not be attributed to a single cause without reproducing the benchmark implementation details.

Possible sources of difference include tokenization, feature settings, seed, class weighting, parameter selection, dataset revision, and score aggregation. TweetEval's task-specific metric and the exact official split must match before a numerical comparison is treated as direct. The TimeLMs and XLM-T studies provide additional benchmark context, but exact settings should be checked in their full methods before quoting scores alongside this result. Huang and Usbeck's 2024 work confirms recent usage of TweetEval but is not treated here as a same-protocol score unless the full split protocol has been verified. This conservative comparison meets the purpose of literature discussion without presenting unlike experiments as identical.

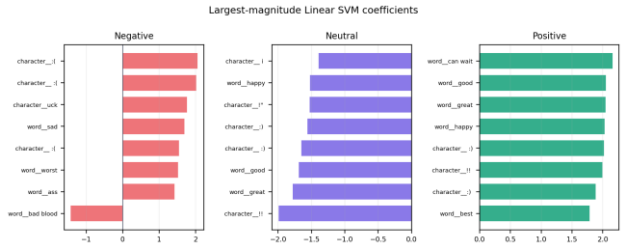


Fig. 6. High-magnitude Linear SVM word and character features by sentiment class. Coefficients indicate learned associations, not causes.

IX. DISCUSSION

A. What the results mean

The experiment shows that a relatively compact classical pipeline can learn useful sentiment patterns from labeled tweets. All three learned models outperform the majority baseline on macro-F1. Word and character n-grams capture enough lexical and orthographic information to distinguish many examples, and the serialized pipeline makes the result easy to demonstrate. For a classroom benchmark, this supports the value of comparing multiple algorithms under the same evaluation protocol.

The result should not be oversold. The selected model's 61.9% accuracy is far below an 80% target. Cross-validation improves the reliability of estimating training-distribution performance and reduces dependence on one fold partition, but it cannot guarantee a high score or solve the inherent ambiguity of the labels. The small difference between validation-selected SVM and Logistic Regression illustrates that the winner is sensitive to the selection metric and sample composition. On the test set, Logistic Regression performs slightly better, while the declared validation selection remains SVM.

The test split's class mix is substantially different from the training and validation splits. This shift is one reason that a model with approximately 0.676 validation macro-F1 produces about 0.614 test macro-F1. It is not appropriate to use the test set to tune class weights or decision thresholds after observing this gap. A future study could specify a separate shift analysis or calibration procedure in advance, but such a study would require a fresh external test set to preserve an unbiased final evaluation.

B. Practical application

The dashboard demonstrates a low-friction workflow for exploratory use. A user can inspect the training, validation, and test distributions, compare model metrics, browse examples, and enter text for inference. In an organization, a similar model might help prioritize a small sample of feedback for review or produce an initial aggregate view. It should not automatically hide, delete, escalate, or act on individual posts. Human reviewers need access to the original context and the ability to correct model output.

The system returns a class label without a calibrated confidence estimate. A label-only output is easy to understand but can be mistaken for certainty. The interface explicitly says the result is a prediction and does not show a confidence percentage. If confidence is later added, it should be calibrated and evaluated rather than treating a raw margin or probability as reliable confidence. Low-confidence examples could be routed to human review, but that policy would need its own evaluation.

C. Limitations

The first limitation is dataset age and provenance. TweetEval was published in 2020, but sentiment examples originate from earlier SemEval collections [1], [3]. Social-media expressions, platform conventions, and topics change. Good performance on this benchmark does not establish performance on present-day posts, other platforms, other languages, or a particular organization's customer messages.

The second limitation is class-distribution shift. The test set has more than twice the negative-class proportion of the training set and roughly half the positive-class proportion. This may reflect the benchmark construction and should be respected rather than "fixed" by resampling the test set. The result is a score for the benchmark's official test data, not a universal estimate of class performance.

The third limitation is feature capacity. TF-IDF n-grams do not model long-range context, word sense, conversational turns, or sarcasm reliably. Character features help with spelling variation but can also learn superficial patterns. A pretrained transformer may capture contextual relationships better, but it adds computational cost, licensing and provenance questions, risk of training-data contamination, and a more complex explanation story.

The fourth limitation is annotation and measurement. Human sentiment labels reflect annotation guidelines and can be subjective. The model is trained to reproduce those labels, not to uncover an objective emotional truth. Accuracy and macro-F1 are useful but do not measure downstream impact, robustness to adversarial phrasing, or fairness across dialects and groups. A single benchmark score cannot establish fairness.

The fifth limitation is data governance. The dataset card lists license information as unknown and the source repository warns about component dataset and platform terms [2], [4]. The prototype should be used in accordance with those

terms. Raw tweets may contain personal or sensitive content even when handles are masked. Reports and presentations should avoid unnecessary quotation of identifiable text, and any deployment should include data minimization and review.

The sixth limitation is literature comparability. Three or more published works use TweetEval, but exact split and scoring equivalence have not been confirmed for every later study considered. The original benchmark and TimeLMs provide clearer comparison points; the full protocols of XLM-T and Huang and Usbeck should be checked before presenting their values as directly comparable. The assignment requirement about a dataset published within 2020–2026 also depends on whether the lecturer refers to the benchmark release or the age of the underlying posts.

D. Improving the work responsibly

Several extensions could be studied without compromising the current evaluation. First, run more extensive model selection using a predeclared nested or repeated CV design on the training split. This would take longer and should not repeatedly adapt to the official test set. Second, compare against a social-media lexicon baseline such as VADER after defining a consistent rule that maps compound scores to three classes. Third, investigate a pretrained Twitter language model, but only after checking license, model provenance, training data overlap, hardware requirements, and whether it has already seen TweetEval or SemEval labels.

Fourth, expand error analysis by sampling false positive and false negative cases from the test set for each class and manually categorizing ambiguity, sarcasm, domain vocabulary, negation, and annotation uncertainty. The sampling method and categories should be decided before inspecting examples to avoid cherry-picking. Fifth, assess external generalization on a separate legally obtained, documented dataset. The external test must remain untouched during model development. Sixth, investigate robustness and subgroup performance only when relevant metadata can be used ethically and lawfully; do not infer protected traits from tweet text to manufacture a fairness analysis.

None of these proposals should be described as already completed. They are future work. The present implementation is a reproducible baseline and a working prototype, not a production service or a system validated for consequential decisions.

E. Ethical considerations

Sentiment labels can be misread as authoritative descriptions of people. The project predicts a dataset label for a text snippet; it does not reveal an author's actual emotional state. The data may contain regional language, dialect, reclaimed terms, or contentious topics. Errors can therefore be unevenly distributed and may be harmful if used for moderation, employment, health, credit, or other consequential decisions. Such uses are outside the validated scope.

The safest use is aggregate, exploratory analysis with human review, transparency about error rates, and a way to challenge or correct output. Dataset and platform terms should be reviewed before redistribution or operational use. The project stores examples in local CSVs and displays them in a local dashboard; the dashboard should not be exposed publicly with raw text without appropriate access controls and rights review.

X. CONCLUSION AND FUTURE WORK

This project designed, implemented, and evaluated a three-class tweet sentiment prototype using the TweetEval benchmark. It documented data provenance, explored class and text characteristics, checked missing values, blank text, labels, and duplicates, constructed word and character TF-IDF features, tuned three model families by stratified three-fold cross-validation, compared them with a majority baseline, and evaluated on the official test split. A local dashboard and saved model demonstrate end-to-end operation.

The validation-selected Linear SVM achieved 61.9% test accuracy and 61.4% macro-F1, substantially exceeding the majority baseline macro-F1 of 21.7%. Logistic Regression was slightly better on the test sample, but was not selected retrospectively. The difference between validation and test results, the distinct class mix, and the model's per-class errors argue against describing the classifier as highly reliable. The project does not achieve 80% accuracy and makes no promise that future tuning will reach it.

The objectives were achieved for a reproducible benchmark prototype and a measured comparison of multiple models. They were not achieved for high accuracy, guaranteed generalization, or verification that the benchmark meets every possible interpretation of the assignment's recency rule. Before submission, the student should confirm dataset acceptance with the lecturer, read the original papers used in the literature review, check the dataset terms, and complete the required AI-use disclosure according to course policy.

Future work should focus on a carefully controlled pretrained social-media model comparison, external evaluation, richer error analysis, and calibration or human-review policies. Any extension should preserve an untouched final evaluation set and report limitations alongside results.

APPENDIX A. REPRODUCIBILITY AND PROJECT FILES

A.1. Reproducing the data and model workflow

The project is organized so that a reader can trace the experiment from acquisition to prediction. The first execution of the data-loading workflow retrieves the sentiment configuration from the public Hugging Face dataset identifier cardiffnlp/tweet_eval. The datasets library may use its local cache on later runs. An Internet connection is therefore needed when the data is not already cached. A successful cached run does not mean that the raw source has become part of the software package or that its terms have changed.

The model workflow is invoked from the workspace root with python -m src.model. This entry point is responsible for loading the dataset, validating expected fields, preserving the official split assignments, producing data summaries, fitting the model searches, evaluating candidates, and writing the outputs under artifacts/. The command-line inference path is python -m src.model --predict "I really

enjoyed this."; it uses the serialized pipeline rather than recreating a vectorizer independently. The interactive demonstration is launched separately with python app.py. Keeping the model and its fitted text features together in one serialized pipeline is essential: loading only the classifier without the exact vectorizer would not reproduce the feature indices on which the model was trained.

The notebook TweetEval_Sentiment_Classification.ipynb is the educational walkthrough. Its order follows the project workflow rather than presenting only the final score: acquire data, examine labels and text, check data quality, create features, define the partitions, fit and compare models, evaluate, and discuss findings. A notebook and command-line run should be treated as two interfaces to the same modeling problem, not as separate experiments with different random splits. When modifying code or data, the saved artifacts should be regenerated together so that figures, CSVs, and the model describe one run.

A.2. Exported data and artifact inventory

The combined export at data/data.csv contains the benchmark examples with a split field identifying training, validation, or test membership. The separate data/train.csv, data/validation.csv, and data/test.csv files make the official partitions convenient to inspect without filtering. They are exports of the named benchmark, not newly collected or independently labeled datasets. The text and label fields are the modeling columns; split membership is bookkeeping that must not be supplied to the classifier.

The artifact directory separates results by purpose. results.json records the selected model and core summary metrics. metrics.csv supports tabular model comparisons, while cross_validation_results.csv records candidate search and fold results. The saved per-class report gives precision, recall, F1, and support for the selected test predictions. test_predictions.csv permits the confusion matrix and paired comparisons to be regenerated. Feature coefficient exports and plots are descriptive views of the fitted linear model. The serialized model contains the fitted vectorizers and classifier. Together, these files provide a trace from model-selection policy to a displayed result.

Artifacts should be interpreted as belonging to the code, data snapshot, and dependency versions used to generate them. A serialized scikit-learn pipeline is not a permanent interchange format guaranteed to load identically in every future release. The current generated report emitted an InconsistentVersionWarning when loading its model artifact because the local scikit-learn installation differed from the version used to serialize it. Loading succeeded for report generation, but this warning should be resolved for reproducible model inference by using a compatible dependency version or retraining and regenerating all model-dependent artifacts in a single pinned environment. Scores already stored as CSV/JSON are reported from those outputs; the warning is not evidence that a new model evaluation was performed.

A.3. Reproducibility boundaries

The fixed random seed helps make shuffled cross-validation folds and stochastic estimator behavior repeatable under a stable software environment. It does not make every low-level operation identical across hardware, library versions, or dataset revisions. Reproduction therefore requires more than copying the seed: the dataset identifier and configuration, official partitions, preprocessing settings, search grids, score definition, package versions, and the chosen model rule all need to remain consistent.

The experiment's selection rule is intentionally explicit. For each model family, grid search maximizes the mean three-fold training macro-F1. The fitted family pipelines are then compared on the official validation set, and the family with the best validation macro-F1 is selected. The official test set is not used to revise that choice. This is why the Linear SVM remains the selected model despite Logistic Regression having a higher test macro-F1 in this run. Re-running the project and selecting according to a different criterion, such as accuracy or the test score, is a new experiment and should be described as such.

An independent reproduction should first compare split sizes and class counts, then compare the selected parameter values, validation scores, and test predictions. If a score differs, investigators should check the actual dataset revision, package versions, random seed, feature vocabulary settings, and the model's class-weight and regularization configuration before attributing the difference to model quality. A single small metric discrepancy should not lead to ad hoc edits to the official test set.

APPENDIX B. METRIC DEFINITIONS AND INTERPRETATION

B.1. Confusion matrix and per-class scores

For three-class classification, a confusion matrix records how many examples of each true label receive each predicted label. In this report, rows correspond to the true class and columns to the predicted class. The diagonal contains correct predictions; an off-diagonal cell represents a specific direction of error. For example, a true positive example predicted as neutral is different from a true neutral example predicted as positive. The class totals are not equal, so inspecting raw cell counts together with row-wise rates is more informative than treating the entire matrix as one number.

For a given class in a one-versus-rest calculation, precision is the number of correctly predicted examples of that class divided by all predictions of that class. Recall is the number of correctly predicted examples of that class divided by all true examples of that class. The F1 score is the harmonic mean of precision and recall, defined as $2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$ when the denominator is nonzero. Precision therefore answers how often a class prediction is correct, while recall answers what fraction of the true class was found. They measure different failure modes and should not be interchanged.

Macro precision, macro recall, and macro-F1 are the arithmetic means of the respective class scores, with each of the three classes receiving equal weight. This is useful here because the class distribution is uneven and the test set has a different composition from training. Macro-F1 can expose weak performance on a less frequent class that would be masked by a score dominated by the larger class. Its equal weighting also means it is not an estimate of the proportion of all future messages that would be correctly classified under an arbitrary population mix.

Accuracy is the number of correct predictions divided by the total number of examples. It is easy to explain, but its value depends on class proportions and can remain deceptively high when one class dominates. The majority baseline's 0.4833 test accuracy alongside 0.2172 macro-F1 illustrates this distinction: predicting a single class gets a substantial fraction right but completely fails to retrieve the other classes. The selected SVM's higher macro-F1 reflects more balanced class recognition, although its accuracy still means that many individual predictions are wrong.

B.2. Cross-validation, selection and uncertainty

Three-fold stratified cross-validation partitions the training split into three folds while approximately preserving class proportions within each fold. For each candidate parameter setting, the estimator is fitted on two folds and scored on the remaining fold; this process rotates the held-out fold so that every training example is scored once. The reported mean is the arithmetic mean of the three fold macro-F1 values. The reported standard deviation describes variation across those three fold scores. It is descriptive and should not be called a 95% confidence interval.

Cross-validation makes better use of the training partition than a single internal holdout, but it does not create three independent datasets from different times or populations. All folds originate from the same benchmark training split, and therefore share its collection process, annotation choices, topic mix, and possible biases. Stratification preserves approximate label shares; it does not correct temporal drift, near-duplicate examples, or source-domain differences. The variation between folds is not a measure of all the ways performance may change in deployment.

The paired bootstrap comparison uses saved test predictions and resamples test-example indices with replacement. For each resample, macro-F1 is computed for both models on the same resampled cases, and the difference is recorded. Using the same resampled indices preserves the paired structure: both systems are evaluated on identical sampled examples. The 2.5th and 97.5th percentiles of 1,000 bootstrap differences form the reported percentile interval. For SVM minus Logistic Regression, the observed difference is negative and the interval is below zero. This is evidence about the relative scores on the sampled benchmark test population, not proof of general superiority on future or current social-media content.

The evaluation reports no calibrated probabilities. Linear SVM's decision scores are not probabilities, and margins from different classes or models should not be presented as confidence percentages without a calibration study. The dashboard therefore displays the predicted class without a numeric confidence. If downstream use requires abstention or confidence thresholds, thresholds must be developed on training/validation data and tested prospectively on a separate held-out sample.

B.3. Why results are not collapsed to one score

A single headline value can hide the practical trade-off among model families. In the held-out test results, Logistic Regression exceeds the selected SVM by about 0.0055 in accuracy and about 0.0080 in macro-F1. On the validation partition, however, the SVM has the highest macro-F1, albeit by only 0.0007 over Logistic Regression. Under the declared rule, the validation macro-F1 winner is the selected model. Showing all three splits prevents a reader from mistaking that prespecified decision for a claim that the SVM is the top scorer under every metric.

The per-class report adds information that the aggregate cannot provide. Positive sentiment has 0.6573 recall but only 0.5510 precision. A system that routes positive predictions to a human moderator or analyst could therefore produce a relatively high number of false-positive positive labels. Conversely, negative recall of 0.5952 indicates that a substantial proportion of negative examples are missed. Whether either error is acceptable depends on the application; the benchmark alone cannot determine the costs. Any deployment decision would need task-specific error costs, an appropriate operating point, and a review process.

APPENDIX C. PROTOTYPE OPERATION AND USER WORKFLOW

C.1. Local dashboard workflow

The dashboard is designed as a local demonstration, not as a hosted service. After starting python app.py, a user opens the local address shown in the project instructions. The first view summarizes the task and key benchmark metrics. Visual panels compare model performance, display split-specific class proportions, show cross-validation variation, and present the selected model's confusion matrix. A data-exploration area allows the user to inspect a bounded sample and filter by split or sentiment. A prediction control accepts a short text and returns one of the three labels.

The intended interaction sequence is: inspect the class mix before interpreting metrics; compare the validation and test summaries; review the confusion matrix and per-class scores; browse a few examples with their benchmark labels; and only then try a new text. This order encourages the user to understand that the model reflects an imbalanced historical benchmark and makes errors. The interface is not intended to encourage a user to treat a single prediction as a fact about a writer.

Illustrative smoke-test inputs include “I absolutely love this, it made my day!”, “The product is okay, nothing special.”, and “Terrible service, I am very disappointed.” The observed outputs were positive, neutral, and negative, respectively. These are hand-selected examples with relatively direct wording. They verify that the interface accepts text, calls the saved pipeline, and displays a label; they do not measure generalization and must not be described as a representative test sample.

C.2. Web dashboard workflow screenshots

The following browser captures document the implemented dashboard workflow: reviewing the overview and benchmark summaries, inspecting evaluation results, and filtering labeled examples in the dataset explorer. The explorer capture shows the test split filtered to positive examples.

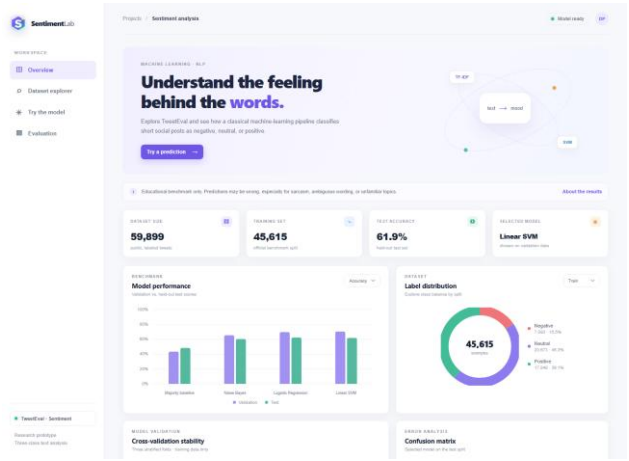


Fig. C.1. Dashboard overview with benchmark summaries and model comparisons.



Fig. C.2. Cross-validation summary and selected-model test confusion matrix.

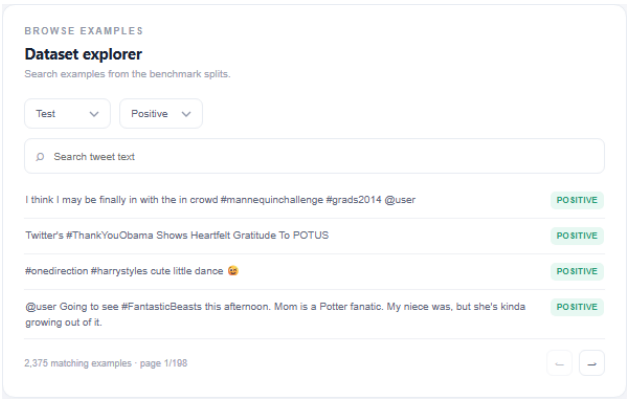


Fig. C.3. Test-set examples filtered to the positive sentiment label.

C.3. Manual prediction test-case screenshots

Each browser capture shows the supplied text and the label returned by the running web interface. These illustrative smoke tests verify the interaction path only; they are not a substitute for held-out test-set evaluation.

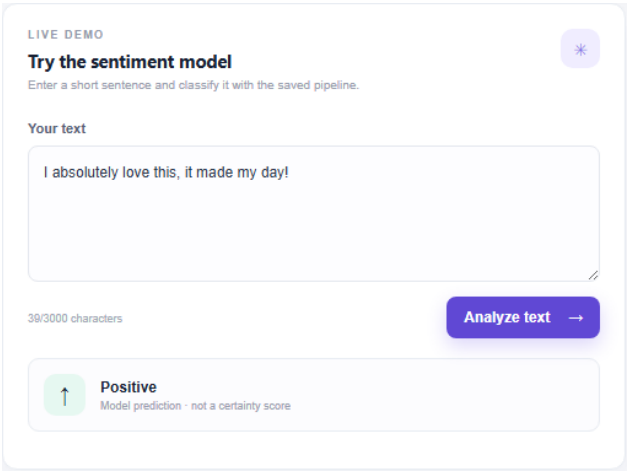


Fig. C.4. Test case 1: “I absolutely love this, it made my day!” returns positive.

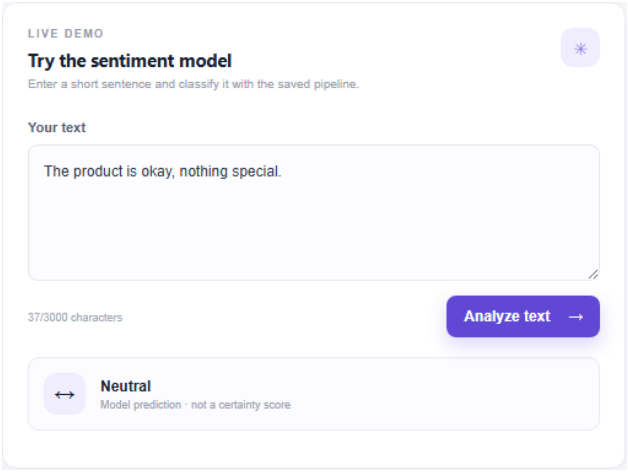


Fig. C.5. Test case 2: “The product is okay, nothing special.” returns neutral.

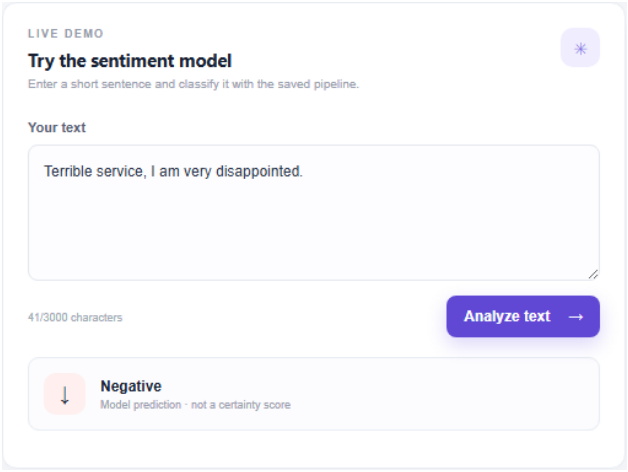


Fig. C.6. Test case 3: “Terrible service, I am very disappointed.” returns negative.

C.4. Expected inputs and outputs

The inference function accepts a text string and returns a predicted sentiment category from the fixed set negative, neutral, and positive. Empty input is rejected or prevented by the interface rather than being interpreted as meaningful sentiment. The model does not request a username, account identifier, timestamp, location, or other user profile information. It operates on the submitted text only. It does not translate other languages or infer the target of the opinion.

The model's output format is deliberately small. It does not return an explanation of causal intent, a calibrated probability, a toxicity rating, or a recommendation to take action. Feature-coefficient plots are aggregate descriptions of what the fitted linear classifier associates with each class. They are not generated as a guaranteed local explanation for a specific dashboard prediction. This distinction reduces the risk that users misinterpret a global coefficient chart as a faithful narrative of one decision.

The system also contains an example-browsing path that is useful in a classroom demonstration but should remain local and appropriately restricted. Tweet text can contain personal or sensitive information even when direct identifiers are not displayed. A publicly exposed dashboard or exported

sample may create data-governance risks that are not addressed by the classroom prototype. Any deployment beyond local demonstration would require a security review, access controls, data-minimization decisions, applicable dataset-term review, and a process for responding to requests or corrections.

C.5. Operational limitations and failure handling

The first uncached dataset download requires network access. A transient network failure, an unavailable hosting service, or a changed upstream dataset can prevent a fresh setup from reproducing the current CSVs. The downloaded and exported project files support offline inspection after acquisition, but a new training run should verify that it uses the same source revision and split sizes. A changed class count is a reason to stop and investigate rather than silently reporting results under the old description.

The saved estimator and current Python environment also need to be compatible. The report-generation run produced a scikit-learn version warning when the saved pipeline was opened under a version other than the one that created it. This was a warning rather than a failed load, but model persistence across library versions is not guaranteed. For a clean hand-off, the model should be regenerated in the declared environment or loaded using its training version, and a smoke test should confirm prediction behavior before showing the dashboard.

The project is a single-user local application and does not include production monitoring, authentication, rate limiting, or a durable inference service. It does not monitor changes in label distribution or feature drift. The model is not automatically updated when current language changes. These are explicit boundaries, not missing features that should be assumed present. A production adaptation would need a new requirements analysis and validation plan.

APPENDIX D. REQUIREMENT TRACEABILITY AND ACCEPTANCE CHECKLIST

D.1. Project requirement traceability

The assignment's data-source requirement is addressed by identifying the Hugging Face TweetEval sentiment configuration, citing its dataset card and original benchmark paper, describing the older SemEval origin, and preserving the official partitions. The residual question is whether the course interprets its publication-window requirement as the benchmark release date or the underlying post-collection date. That point cannot be settled by the implementation and should be confirmed with the lecturer.

The data-understanding requirement is addressed through the field description, label mapping, partition sizes, class distribution, character-length summary, and visualizations. The preprocessing requirement is addressed by checking missing and blank text, invalid labels, and exact within-split duplicates, then explaining why official training duplicates were retained. The feature-engineering requirement is met by describing and implementing word and character TF-IDF n-grams within a pipeline. The split requirement is addressed by preserving the supplied benchmark training, validation, and test sets and explaining each split's role.

The model-selection and training requirements are addressed through a majority baseline, Multinomial Naive Bayes, Logistic Regression, Linear SVM, three-fold stratified cross-validation, explicit parameter grids, and validation-based family selection. The evaluation requirement is addressed by reporting accuracy, macro precision, macro recall, macro-F1, per-class scores, a confusion matrix, and a paired bootstrap comparison. The results and discussion requirements are addressed by interpreting observed values, comparing with a baseline, describing the validation/test ranking change, and explaining why performance does not justify an 80% accuracy claim.

The implementation requirement is addressed by the notebook, exported CSV files, saved model and result artifacts, command-line prediction path, and local dashboard. The report includes a system architecture figure and a description of the user path. The ethics and limitations discussion identifies historic data, distribution shift, annotation uncertainty, unknown dataset license status, potential harms, and the need for human oversight. The bibliography contains more than ten scholarly sources, but citations still require review by the student against the original publications before submission.

D.2. Submission-readiness checklist

Before a final course submission, the student should open the notebook in the chosen environment and run it from the first relevant cell through its final results, or provide a clear explanation for any cached outputs. The student should compare notebook outputs with the generated CSV and JSON artifacts and confirm that all metric values in the report match the same saved run. A report that mixes outputs from different runs can be internally inconsistent even if each individual number once appeared in a valid experiment.

The student should verify that the PDF meets the exact page-count rule when rendered using the required template and software. Pagination can vary by Word version, fonts, printer settings, and PDF conversion. The report should be visually inspected at normal zoom for clipped figures, broken tables, orphaned headings, incorrect column transitions, and missing page numbers. The supplied conference template is preserved in the workspace; this generated report uses its A4, Times New Roman, 10-point, two-column conference-style conventions but is not a byte-for-byte edit of the supplied template file.

The student should check every citation in the source material, confirm title, author order, venue, year, page numbers, and DOI or URL, and ensure each source supports the specific statement attached to it. The bibliography deliberately warns when split equivalence is unverified. Such caveats should not be removed merely to make results appear more comparable. The student should also confirm that the dataset's current terms permit the planned course use and that any quoted example text is appropriate to redistribute.

Finally, the required AI-use declaration should accurately describe the support received and follow the institution's policy. The current report text discloses AI-assisted drafting and editing and requires the student to verify sources, code, and outputs, understand the analysis, and revise the report in their own understanding. That disclosure is not a substitute for the student's own academic responsibility. Personal

details, course information, and submission metadata should be checked against the official course records before submission.

APPENDIX E. EXTENDED ERROR ANALYSIS AND NEXT EXPERIMENT

E.1. Reading the observed errors

The current saved predictions support quantitative analysis of which labels are confused, but the report does not claim to have completed a systematic human annotation of the error causes. The confusion matrix identifies the direction and frequency of mistakes; it does not reveal whether a specific error resulted from sarcasm, an ambiguous phrase, a topic term, a data-quality issue, or a label disagreement. Assigning a linguistic explanation to all errors based only on aggregate metrics would be speculation.

A careful follow-up should define a sampling protocol before reading examples. One workable design is to sample a fixed number of false positives and false negatives for each class from the official test predictions, using a recorded random seed and ensuring that examples are not duplicated across categories. Two reviewers could independently apply a short coding guide with categories such as negation, sarcasm or irony, mixed sentiment, topic-specific vocabulary, spelling or tokenization, neutral-versus-polar ambiguity, and apparent annotation mismatch. Reviewers should also be allowed to mark an example as unclear or outside the taxonomy rather than forcing a category.

The coding guide should define each category with inclusion and exclusion rules. For example, a negation tag should require a negating construction that plausibly changes polarity, not merely the presence of the token “not.” Sarcasm should be marked only when there is textual or contextual evidence rather than because a model was wrong. Mixed sentiment should distinguish praise and criticism in the same message from neutral language that simply mentions an opinionated subject. These rules reduce the risk that a reviewer uses the model's error as evidence for a preferred interpretation.

Inter-reviewer agreement should be reported before disagreements are reconciled. Agreement statistics do not establish that the categories are objectively true, but they show how consistently the guide can be applied. Reviewers can discuss disagreements after independent coding and document the resolution. If annotation is performed on benchmark examples, the process should follow dataset terms and avoid reproducing raw posts unnecessarily. Redacted or paraphrased examples may help presentations, but paraphrase must be clearly labeled because it is no longer the original input that produced the prediction.

The current report's per-class numbers suggest questions for such a study rather than conclusions about causes. Positive precision is weaker than positive recall, so review could test whether positive predictions are over-triggered by generally upbeat vocabulary or whether the benchmark's test distribution contributes to that pattern. Negative recall is below 0.60, so a separate sample of missed negative examples could explore whether subtle criticism, negation, or neutral-looking factual statements are involved. A review protocol can investigate these hypotheses without changing the reported test metrics or claiming a cause in advance.

E.2. A leakage-conscious follow-up study

A next modeling experiment should state its question before opening the test predictions. If the question is whether contextual language models improve macro-F1 over the TF-IDF baseline, the comparison should specify model checkpoints, preprocessing, training budget, seed policy, and whether benchmark examples may have appeared in pretraining or supervised fine-tuning. Training compute, inference latency, memory use, model licensing, and provenance should be recorded alongside quality scores; a higher score does not automatically make a model more appropriate.

For a modest extension, candidate models and parameter ranges can be evaluated using training data only, with the official validation partition reserved for a final choice. If repeated comparisons are made against the validation set, the researcher should recognize that decisions can gradually overfit that set. A new external test set is preferable for a claim about current language or a different platform. It should be collected under applicable terms, labeled with a documented guideline, screened for duplicates against training material where possible, and held untouched until the model and threshold are fixed.

When the goal is to estimate variability from data splitting, repeated stratified cross-validation may be used within the training data, but its computational cost and dependence between folds must be explained. Nested cross-validation can separate hyperparameter selection from performance estimation more rigorously, although it requires substantially more model fits. Neither method repairs a time shift or substitutes for evaluation on a genuinely later sample. The methodology should choose the least complex design that answers the new question rather than adding cross-validation merely to claim that evaluation is stronger.

If an organization has a changing deployment population, a temporal evaluation is more informative than a random holdout. Training on earlier examples and evaluating on a later time window can expose vocabulary drift and shifts in topic or class prevalence. However, temporal evaluation is only meaningful when reliable timestamps and collection provenance are available. The current exported feature schema and published benchmark should not be treated as if it support such an analysis when those fields are absent.

E.3. Responsible thresholds and human review

The project produces a single class prediction and does not provide a tested abstention mechanism. A practical workflow could add a human-review queue for uncertain or high-impact cases, but it first needs a meaningful uncertainty measure. For Logistic Regression, probabilities may be available from the fitted estimator, though their calibration still needs testing. For a linear SVM, decision margins can be considered for ranking ambiguity, but a margin is not a probability. Calibration must use data separate from the final test evaluation.

Thresholds should be chosen using the actual cost of false positives and false negatives in the target task. For example, an aggregate product-feedback dashboard may tolerate more false-positive positive labels than a system used to prioritize urgent negative feedback. The acceptable balance cannot be inferred from benchmark macro-F1 alone. A threshold

policy should specify who reviews flagged examples, how review decisions are recorded, how errors are corrected, and how model changes are evaluated.

Human oversight is meaningful only if reviewers have enough context and authority to disagree. A user interface that displays one bold sentiment badge without the underlying message context can create automation bias, even if a disclaimer is present. Reviewers should be shown that predictions can be wrong, receive an explanation of the benchmark's limitations, and have a clear path to disregard the output. The present dashboard is an educational prototype and has no such operational governance process.

E.4. Monitoring and retirement criteria

If the model were adapted for an approved low-risk exploratory workflow, monitoring should begin with simple data-quality and distribution checks, not automatic retraining. Useful signals include the share of each predicted class, rates of empty or out-of-range input, changes in text length or character patterns, and periodic human-reviewed error samples. A change in predicted class proportions is a prompt for investigation; it is not proof of model drift because the underlying population may also have changed.

Performance monitoring requires fresh labeled examples obtained with consent and appropriate governance. The labels should be independent of the model output where practical, and the evaluation window should be recorded. If reviewers label only examples the model considers uncertain, the resulting data is selection-biased and cannot provide a simple unbiased accuracy estimate without an appropriate sampling design. Monitoring must also consider whether errors vary across language varieties or relevant user groups, but such analysis should rely on ethically obtained and valid metadata rather than inferred sensitive traits.

An operational owner should define conditions under which the system is paused, reviewed, or retired. These could include a substantial verified rise in error rates, a material change in source terms, loss of package compatibility, inability to reproduce the current model, or a change in intended use. No monitoring, retraining, or retirement automation is implemented in this project. Stating these requirements helps distinguish a demonstration model from a maintained system.

APPENDIX F. INTERPRETATION OF THE MODEL COMPARISON

F.1. Baseline as a sanity check

The majority baseline is important because a three-class classifier can appear successful if it is evaluated only by accuracy on an imbalanced dataset. On the training split, neutral is the largest class, and the baseline therefore predicts neutral for every message. On the test split, neutral remains the largest class at 48.3%, so the baseline obtains 0.4833 accuracy. Yet its macro-F1 is only 0.2172 because its recall for negative and positive classes is zero, which results in zero F1 for those classes under the standard class-wise calculation.

The difference between this baseline and the learned models is evidence that the text features contain predictive signal under the benchmark labels. It does not establish that the model has learned robust semantic understanding. It may rely on lexical correlations, punctuation, spelling, or topic cues that change in other settings. A useful baseline is therefore a lower bound for comparison, not a production acceptance threshold.

The Naive Bayes classifier yields 0.6045 test accuracy and 0.6010 macro-F1. It improves materially over the baseline, but scores below the two linear discriminative models. Additive smoothing prevents zero likelihoods for unseen features, while the conditional independence assumption is a simplification of language. Its performance indicates that a comparatively simple probabilistic model is a reasonable reference, but the current experiment does not compare it with a tuned lexicon model such as VADER.

F.2. Linear models and the validation decision

Logistic Regression reaches the highest mean cross-validation macro-F1 at 0.6536, while Linear SVM is close at 0.6513. On the validation set, their macro-F1 values are 0.6758 and 0.6765, respectively. The SVM's advantage is seven ten-thousandths, a small difference that should be understood as the outcome of a deterministic selection rule rather than a practically decisive margin. The validation scores are rounded in the report; the code should be consulted for full precision if a tie-breaking decision is being audited.

On the held-out test data, Logistic Regression records 0.6246 accuracy and 0.6223 macro-F1, compared with 0.6191 and 0.6143 for SVM. The paired bootstrap interval for the SVM-minus-Logistic macro-F1 difference is approximately [-0.0126, -0.0032]. This comparison makes the test ranking clear, while the model remains the validation-selected SVM according to the procedure established before test evaluation. There is no contradiction: selection and final comparison answer different questions.

The validation/test gap may be related to the shift in class composition, but it cannot be attributed exclusively to class priors from the reported evidence. Differences in examples, ambiguity, topics, and sample size also contribute. Correcting class weights or decision thresholds after seeing test results would invalidate the test as a final estimate. A future experiment may examine prior-shift adjustment, but it must define the method and use an independent evaluation set.

F.3. Limits of coefficient inspection

The linear feature plot displays features with large absolute coefficients. Since word and character features occupy one combined sparse space, the plot may include word fragments or character sequences that are not meaningful as standalone words. The magnitude depends on feature scaling and the fitted model. A positive coefficient for a class indicates an association in the multiclass decision function; it does not mean that the feature always signals that class or that the model's behavior is causally explained.

Correlated n-grams can share or redistribute weight. The coefficient of one word may change when the vocabulary, minimum frequency, or regularization changes even when overall scores remain similar. The plot is therefore most useful as a qualitative inspection aid: it can reveal obviously surprising associations, confirm that some intuitive signals appear, and motivate a more systematic audit. It should not

be used to infer what an individual author intended or to claim that a particular word caused a prediction.

Instance-level explainability would require a different analysis. A method such as perturbation-based feature attribution could estimate which input features affect a particular prediction, but it would require careful validation for sparse text and should be checked for stability. Such an explanation remains an approximation of model behavior rather than a causal account. No local-attribution interface is implemented here.

F.4. Meaning of the current accuracy level

The selected SVM gets approximately 62 of every 100 official test examples correct. The remaining errors are too common for the prediction to stand alone in a decision about an individual message. This rate is not a failure of cross-validation; cross-validation estimates performance on training-like folds and supports parameter choice, while the final score shows what happened on the different official test split. No evaluation method can guarantee a target accuracy simply by being repeated more often.

The requested aspiration of 80% accuracy is not reached. More extensive tuning may alter results, but it may also overfit the validation data and cannot be promised to close a gap of this size. Domain-specific pretraining, a better matching dataset, additional context, annotation review, or a different task definition could improve a future system, but these changes would require new experiments. The present report records measured performance rather than presenting an untested improvement as achieved.

APPENDIX G. DATA INTERPRETATION AND SCOPE OF VALID CLAIMS

G.1. What the dataset labels represent

The target values in TweetEval are benchmark sentiment annotations. They are not direct measurements of a writer's private emotional state. A message labeled positive is an example assigned to the positive class under the source annotation process and benchmark mapping. It does not establish that the author felt happy, approved of every entity mentioned, or intended the text literally. Similarly, a negative label is not a diagnosis or a measure of risk. The model's formal task is to reproduce a categorical label from the text, not to infer a psychological condition.

The three-way label scheme also compresses a richer space of communication. Text can express approval of one aspect and disapproval of another, uncertainty, humor, sarcasm, factual reporting, or a reaction whose target is not obvious from the message alone. The classification target requires one output class for each record, even when a human reader would prefer multiple labels or an abstention. This mismatch between simple output and nuanced communication is a structural limitation of the task, not something that a larger vocabulary alone can eliminate.

Neutral should not automatically be interpreted as an absence of emotion. In a benchmark, it is a defined class that can include text judged not clearly positive or negative under the task rules. It may include factual language, questions, mild opinions, or context-dependent statements. A model that predicts neutral is therefore reporting its best class under the trained decision boundary, not asserting that the author had no sentiment.

G.2. Provenance and temporal context

Dataset provenance matters because the meaning of model performance depends on how examples were assembled. TweetEval provides a common benchmark release, and the report names both that release and the SemEval sentiment source. The benchmark's age does not guarantee that the language is current. Slang, platform conventions, and public topics change, while the training vocabulary and learned associations remain fixed after fitting. Even if the evaluation split was held out from the training code, it belongs to the same benchmark construction and does not represent a later deployment population by default.

The report therefore uses the phrase “official TweetEval test split” for its final dataset evaluation. It does not call that result a current-year test or an evaluation of current social media. A benchmark comparison and a modern deployment test are different evidence. To support a claim about present-day use, a researcher would need a documented later sample collected under appropriate terms, a clear annotation process, and an evaluation protocol that prevents the new data from influencing model choice.

The labels and fixed partitions are useful for reproducibility, but they do not remove every source of sampling bias. The observed test class mix is different from the training and validation mixes. The model's test metrics consequently summarize performance on that specified split. They cannot be converted to a universal operational accuracy without assumptions about the future class frequencies and error behavior. Reweighting metrics could answer a defined hypothetical population question, but it would need a justified target distribution rather than a guess.

G.3. Text transformations and privacy

The preprocessing intentionally avoids aggressive cleaning. Lowercasing and the vectorizers' analyzer settings are part of feature extraction, while the original exported text is retained. This allows a user to inspect what was modeled and avoids manual rewriting that could erase polarity cues. At the same time, retaining original text means the CSVs and dashboard can expose content that may be personal or sensitive. A local classroom artifact should be stored and shared with the same care as other data containing user-generated text.

Hashtags, handles, URLs, punctuation, and placeholders can be useful signals to a classifier, but their presence also creates risks. The model could learn correlations associated with a particular event, account style, or annotation artifact rather than a general sentiment rule. If the data includes anonymization placeholders, the model may respond to those conventions. A future audit could compare performance with selected token categories masked, but such masking would change the feature pipeline and should be treated as a new controlled experiment.

The model does not need author identity to produce its class prediction. Collecting, inferring, or adding identity and profile information would expand the privacy surface without being required by this task. This design choice does not make the underlying text anonymous; people may sometimes be identifiable from their words or context. For

that reason, the project limits the intended use to aggregate exploration and education and avoids presenting identifiable sample messages in the report.

G.4. Claims supported and not supported

The evidence supports the following bounded statements: the public TweetEval sentiment configuration contains the recorded official partitions; the implemented pipeline compared specified classical models with a majority baseline; training-fold cross-validation selected candidate hyperparameters; validation macro-F1 selected Linear SVM; and the saved test predictions produce the reported metric values. The notebook, generated artifacts, tests, and local dashboard provide a reproducible educational implementation of that workflow, subject to dependency and dataset availability.

The evidence does not support claims that the classifier understands sentiment in a human sense, performs at 80% accuracy, generalizes to current social platforms, is unbiased across dialects, is safe for moderation decisions, or can identify a person's actual emotion. It does not establish that a 2020 benchmark release resolves every interpretation of the course recency requirement, nor that every cited later study used exactly the same test split. The report treats these as open questions rather than quietly converting them into positive claims.

Keeping claims proportionate to the evidence is part of the technical result. A moderate model score can still support a successful coursework project if the method is clearly specified, the evaluation is honest, the prototype works, and the limitations are understood. The principal value here is a transparent experiment with explicit boundaries, not an assertion that a simple classifier is ready to make consequential judgments.

APPENDIX H. REPRODUCIBLE DEMONSTRATION RUNBOOK

H.1. Preparing a clean demonstration

The demonstration should start from the project root so that relative paths to data/, artifacts/, and web/ resolve as expected. Before running the model, the student should review the dependency file and use a compatible Python environment. The model artifact records a scikit-learn version that differs from the currently installed version in the report-generation environment. A clean demonstration should avoid relying on silent compatibility assumptions: either install the version used to create the artifact or retrain the model in the current environment and regenerate the associated metrics and plots. The selected environment and the fact of retraining should be recorded.

If an environment is recreated, the dataset and dependency installation steps should be performed once and their output checked for errors. Network access is required when the TweetEval data is not cached. The student should not proceed to a live dashboard if model loading failed, if the expected split files are missing, or if the model version warning has not been understood. A successful browser page is not sufficient evidence that a compatible classifier is serving the request; prediction smoke tests are needed as well.

H.2. Verifying data and evaluation artifacts

After data acquisition, compare the exported row counts with the stated official sizes: 45,615 training, 2,000 validation, and 12,284 test examples. Verify that the combined export's split field contains only the expected partition names and that each separate CSV contains examples from only its named partition. Check that the target labels map consistently to negative, neutral, and positive. These checks catch accidental concatenation, filtering, or label-remapping mistakes before any score is reported.

Next inspect the generated summary artifacts. The class distribution CSV should agree with the partition sizes and the proportions in the report. The metrics table should identify one row per evaluated model, and the cross-validation table should show the tuned results for the candidate learned models. The selected model name in the results JSON should agree with the validation macro-F1 comparison. The per-class report and test-predictions file should cover the same test count. If these artifacts disagree, the user should regenerate them from one run rather than editing one output manually.

The final test scores may vary only if the data, implementation, or environment has changed. A mismatch should be investigated through recorded parameter choices, dependency versions, and the selected model artifact. The test set must not become a tuning set simply because one result differs from an earlier run. If an implementation change is made after observing test results, the report must identify that change and obtain a fresh independent evaluation set for a new final performance claim.

H.3. Demonstrating the notebook

The notebook presentation should make the flow visible in a sequence. Begin with the project question and source attribution. Show the split sizes and class proportions before describing the model, because the composition helps explain later metrics. Then show the data-quality checks and explain the decision to retain official training duplicates. Next show how the word and character vectorizers are included in the cross-validation pipeline, why they must be fitted separately inside folds, and how the validation partition is used after the training search.

When presenting results, distinguish the mean cross-validation score from the validation model-selection score and the final test score. State that the Linear SVM is selected because it narrowly wins validation macro-F1; then note that Logistic Regression has the higher test score without changing the original choice. This is a useful demonstration of evaluation discipline. Avoid describing the highest number shown anywhere as the selected result without explaining which split produced it.

The presentation should finish with the per-class report, confusion matrix, error and distribution-shift discussion, limitations, and a prediction example. A direct demonstration can show one positive, one neutral, and one negative illustrative input, but the user should say explicitly that these are smoke tests. A useful closing question is whether a 62% benchmark result is sufficient for a consequential use; the answer for this prototype is no

without a different evidence base, human review, and a more suitable evaluation.

H.4. Demonstrating the dashboard

For the local interface, run the documented startup command and open the local address in a browser on the same machine. Start with the metrics and dataset panels rather than immediately entering a prediction. Select different splits in the dataset browser and confirm that the displayed labels and sample counts correspond to the selected partition. Review the model comparison and cross-validation visualizations, then inspect the confusion matrix to connect the numeric metrics to the types of class errors.

Use a short text example with clear polarity, then one ambiguous or mixed-polarity example. The first helps confirm the request-response path; the second demonstrates that real language can be difficult even when the code works. A surprising output should be treated as a model error or ambiguity to discuss, not corrected by inventing an interpretation after the fact. The dashboard does not provide a numeric confidence score, and the presenter should not infer one from the displayed label.

At the end of the demonstration, stop the local server and note that it is a local educational prototype. The application is not authenticated for public access, is not monitored, and does not implement a review queue. Exposing it beyond the local machine changes its risk profile and requires additional engineering. Any failure to start, data-loading error, or model-loading warning should be reported accurately; screenshots should not be used to imply that an unresponsive or stale interface is live.

H.5. Versioning and change log discipline

Any later modification should record what changed and which outputs were regenerated. A change to vectorizer parameters can change the learned vocabulary and invalidates the old serialized pipeline. A change to the training data requires retraining, new validation selection, a new final test evaluation, and refreshed figures. A dashboard-only styling change need not change model metrics, but the model artifact should still be checked when the UI loads it.

The generated report and its PDF should be treated as snapshots. If a source paragraph or result changes, regenerate both document formats and compare them. Do not update a metric in prose while leaving an old chart or JSON artifact in place. Similarly, do not redraw a plot from a different artifact version and assume that its model identity still matches the selected model. Coherent versioning is a small but important part of a trustworthy demonstration.

H.6. Evidence hierarchy for the presentation

For claims about actual classifier performance, use the saved test metrics and prediction-derived analysis, and make the split explicit. For claims about implementation behavior, use the executed notebook, automated tests, and direct dashboard checks. For claims about dataset size, labels, provenance, and licensing, use the current dataset card and source papers rather than relying solely on this report. For claims about a published study's score or exact split, consult the paper and its experimental setup directly.

If sources disagree or a detail cannot be verified, preserve the uncertainty in the presentation. The project report already distinguishes benchmark release date from original post age, and benchmark use from confirmed split equivalence. Removing those distinctions could make a report sound more confident but would make the evidence weaker. A credible demonstration explains what is known, what was measured, and what still requires confirmation.

ACKNOWLEDGMENT AND AI-USE DISCLOSURE

The project report was prepared with AI-assisted drafting and editing support. The student must verify every statement against the code, data, and original cited works; ensure they can explain the methods and results; revise the text in their own understanding; and complete the institution's required AI-use declaration accurately before submission. No claim is made that the assistant personally read sources not inspected by the student.

REFERENCES

[1] F. Barbieri, J. Camacho-Collados, L. Espinosa Anke, and L. Neves, “TweetEval: Unified benchmark and comparative evaluation for tweet classification,” in Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1644–1650, 2020, doi: 10.18653/v1/2020.findings-emnlp.148.

[2] Cardiff NLP, “TweetEval dataset card: sentiment configuration,” Hugging Face Datasets. [Online]. Available: https://huggingface.co/datasets/cardiffnlp/tweet_eval. Accessed: Oct. 18, 2026.

[3] S. Rosenthal, N. Farra, and P. Nakov, “SemEval-2017 task 4: Sentiment analysis in Twitter,” in Proc. 11th International Workshop on Semantic Evaluation (SemEval-2017), pp. 502–518, 2017, doi: 10.18653/v1/S17-2088.

[4] Cardiff NLP, “TweetEval: unified benchmark and comparative evaluation for tweet classification,” GitHub repository and test leaderboard. [Online]. Available: https://github.com/cardiffnlp/tweeteval. Accessed: Oct. 18, 2026.

[5] D. Loureiro, F. Barbieri, L. Neves, L. Espinosa Anke, and J. Camacho-Collados, “TimeLMs: Diachronic language models from Twitter,” in Proc. 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, pp. 251–260, 2022, doi: 10.18653/v1/2022.acl-demo.25.

[6] F. Barbieri, L. Espinosa Anke, and J. Camacho-Collados, “XLM-T: Multilingual language models in Twitter for sentiment analysis and beyond,” in Proc. 13th Language Resources and Evaluation Conference, pp. 258–266, 2022. [Online]. Available: https://aclanthology.org/2022.lrec-1.27/.

[7] B. Pang and L. Lee, “Opinion mining and sentiment analysis,” Foundations and Trends in Information Retrieval, vol. 2, nos. 1–2, pp. 1–135, 2008, doi: 10.1561/1500000011.

[8] B. Pang, L. Lee, and S. Vaithyanathan, “Thumbs up? Sentiment classification using machine learning techniques,” in Proc. ACL-02 Conference on Empirical Methods in Natural Language Processing, pp. 79–86, 2002, doi: 10.3115/1118693.1118704.

[9] C. Hutto and E. Gilbert, “VADER: A parsimonious rule-based model for sentiment analysis of social media text,” in Proc. International AAAI Conference on Web and Social Media, vol. 8, no. 1, pp. 216–225, 2014, doi: 10.1609/icwsm.v8i1.14550.

[10] J. Huang and R. Usbeck, “Revisiting supervised contrastive learning for microblog classification,” in Proc. 2024 Conference on Empirical Methods in Natural Language Processing, pp. 15644–15653, 2024, doi: 10.18653/v1/2024.emnlp-main.876.

[11] D. Q. Nguyen, T. Vu, and A. T. Nguyen, “BERTweet: A pre-trained language model for English tweets,” in Proc. 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 9–14, 2020, doi: 10.18653/v1/2020.emnlp-demos.2.

[12] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” Information Processing & Management, vol. 45, no. 4, pp. 427–437, 2009, doi: 10.1016/j.ipm.2009.03.002.

[13] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why should I trust you?’: Explaining the predictions of any classifier,” in Proc. 22nd ACM SIGKDD

International Conference on Knowledge Discovery and Data Mining, pp. 1135–1144, 2016, doi: 10.1145/2939672.2939778.